

Casos de Teste - API

DATA	DESCRIÇÃO	AUTOR(A)
11/05/2025	Criação da documentação de Plano de Testes	Érika Santana
12/05/2025	Finalizando atualização do documento	Érika Santana

1 - Descrição do Sistema

API <https://reqres.in/> para testes

1.2 - Informações Adicionais

2 - Caso de Teste -Listar usuários e validar dados

Funcionalidade	Realizar a validação da resposta 200 e verificar o conteúdo de dados corretos.
Pré-Condições	Requisição GET
Caminho	https://reqres.in/api/users?page=2

Descrição	Roteiro de teste	Resultado esperado	Observações
1. Validar status 200 e verificar retorno de usuários (id, first_name, last_name, email)	- Realizar uma requisição GET para o endpoint /api/users?page=2. - Validar se o status HTTP da resposta é 200. - Converter a resposta em JSON. - Validar se o campo "data" é um array. - Percorrer e validar cada usuário retornado. - Verificar se o user id é um número. - Verificar se os users são strings.	Sucesso	

2. Cenário adicional: validar estrutura de dados com Assertions (ex: garantir que email tenha um formato válido).	- Validar o formato do email com regex.	Sucesso	
---	---	---------	--

- **Ferramenta:**

- **Código:**

```
import { test, expect, request } from '@playwright/test';

/*Teste para listar usuários e validar os dados da resposta*/
test('Deve listar usuários da página 2 e validar estrutura dos dados', async ({ req

    /*Faz uma requisição GET para o endpoint*/
    const response = await request.get('https://reqres.in/api/users?page=2');

    /*Valida se o status HTTP da resposta é 200 (OK)*/
    expect(response.status()).toBe(200);

    /*Converte a resposta em JSON*/
    const responseBody = await response.json();

    /*Valida se o campo "data" é um array*/
    expect(Array.isArray(responseBody.data)).toBeTruthy();

    /*Percorre cada usuário retornado para validar os campos*/
    for (const user of responseBody.data) {
        /*Verifica se o ID existe e é um número*/
        expect(user).toHaveProperty('id');
        expect(typeof user.id).toBe('number');
```

```

/*Verifica se o primeiro nome existe e é uma string*/
expect(user).toHaveProperty('first_name');
expect(typeof user.first_name).toBe('string');

/*Verifica se o sobrenome existe e é uma string*/
expect(user).toHaveProperty('last_name');
expect(typeof user.last_name).toBe('string');

/*Verifica se o email existe e é uma string*/
expect(user).toHaveProperty('email');
expect(typeof user.email).toBe('string');

/*Valida o formato do e-mail com regex*/
const emailRegex = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+$/;
expect(user.email).toMatch(emailRegex);
}

});

```

2.1 - Caso de Teste - Criar e atualizar um usuário

Funcionalidade	Realizar a criação e atualização de usuário
Pré-Condições	Requisição POST e PUT
Caminho	https://regres.in/api/users

Descrição	Roteiro de teste	Resultado esperado	Observações
1. Criação de usuário	- Definir os dados do usuário a ser criado. - Enviar a requisição POST para criar o usuário. - Incluir a chave de autenticação. - Definir o corpo da requisição com os dados do novo usuário. - Validar se o status da resposta é 201 (Created). - Converter a resposta para JSON para acessar os dados retornados. - Verificar se o	Sucesso	

	nome retornado é o mesmo que foi enviado.		
2. Atualizar dados usuário	- Faça uma requisição PUT para o endpoint /api/users/2 e valide se o usuário foi atualizado corretamente.	sem	
3. Cenário adicional: Validar tempo de resposta	- Valide se os tempos de resposta da API estão dentro de limites aceitáveis.	sem	

- **Ferramenta:**

- **Código:**

```

/*Teste POST*/
test('Deve criar um novo usuário com sucesso (POST /api/users)', async ({ request }) => {
  /*Define os dados do usuário a ser criado*/
  const newUser = {
    name: 'Maria',
    job: 'Teste',
  };

  /*Envia a requisição POST para criar o usuário*/
  const response = await request.post('https://reqres.in/api/users', {
    /*incluir a chave de autenticação*/
    headers: {
      'x-api-key': 'reqres-free-v1',
      'Content-Type': 'application/json'
    },
  }, {
    /*Define o corpo da requisição com os dados do novo usuário*/
    data: newUser,
  });

  /*Valida se o status da resposta é 201 (Created)*/
  expect(response.status()).toBe(201);
}

```

```

/*Converte a resposta para JSON para acessar os dados retornados*/
const responseBody = await response.json();

/*Verifica se o nome retornado é o mesmo que foi enviado*/
expect(responseBody.name).toBe(newUser.name);
expect(responseBody.job).toBe(newUser.job);

});

```

2.2 - Caso de Teste - Manipulação de falhas na API

Funcionalidade	Realizar a validação de erro e falha de respostas da API
Pré-Condições	- Teste DELETE
Caminho	https://reqres.in/api/users/999

Descrição	Roteiro de teste	Resultado esperado	Observações
1. Validação de resposta 404	- Faça uma requisição DELETE para um usuário que não existe (/api/users/999) e valide se a resposta correta de erro (404) é retornada.	Sucesso	
2. Verificar se o sistema lida com erro corretamente	- Simule um cenário de falha de rede ou tempo limite na API e verifique se o sistema lida com o erro corretamente.	sem	

- **Ferramenta:**
- **Código:**

```

/*DELETE*/
test('Deve retornar erro 404 ao tentar deletar um usuário inexistente (DELETE /a
const response = await request.delete('https://reqres.in/api/users/999', {
  headers: {
    'x-api-key': 'reqres-free-v1',
    'Content-Type': 'application/json'
  }
});

```

```
    },  
    });  
  
    /*Valida se o status da resposta é 404 (Not Found)*/  
    expect(response.status()).toBe(404);  
  
    });
```